STM32 + W5500 网络线缆测试仪固件设计

Oct 14, 2026 · @harry

概述与默认假设

设备通过 W5500（SPI 转以太网）接入 LAN，自动 DHCP 获取 IP，在 TCP 5025 端口提供标准 SCPI 服务，任何人（包括 AI agent）都能远程控制和调试线缆测试仪。固件分为 Hardware → Driver → Service → App 四层，运行在 FreeRTOS 上，每一层都可单独自检和测试。

本文在信息缺失处采用以下默认值，确认后可直接修改：

项目	默认假设	说明
MCU	STM32G071RB（NUCLEO-G071RB，Cortex-M0+，64 MHz，128 KB flash，36 KB RAM）	已确认；资源较紧，见下方 G071 注意事项
W5500 接口	SPI1 + DMA（PA5 SCK / PA6 MISO / PA7 MOSI，即 Arduino D13 / D12 / D11），CS / RST 为 GPIO，INT 为 EXTI	起步 8–16 MHz，G071 上限 32 MHz（PCLK/2）
被测线缆	暂无测试仪硬件；第一阶段用 SCPI 字符串命令和模拟 tester_io 验证，之后按 8 芯（如 RJ45 Cat5e/Cat6）设计	通过 `CONFigure:PINS` 可改
SCPI 端口	TCP 5025（raw socket）	业界 SCPI over LAN 惯例端口
DHCP 失败回退 IP	192.168.1.200 / 255.255.255.0	可通过 SCPI 修改并保存到 flash
第三方库	WIZnet ioLibrary_Driver、j123b567/scpi-parser	均为开源、成熟方案
RTOS	FreeRTOS（CMSIS_V2），tick 1 kHz	CubeMX 自带

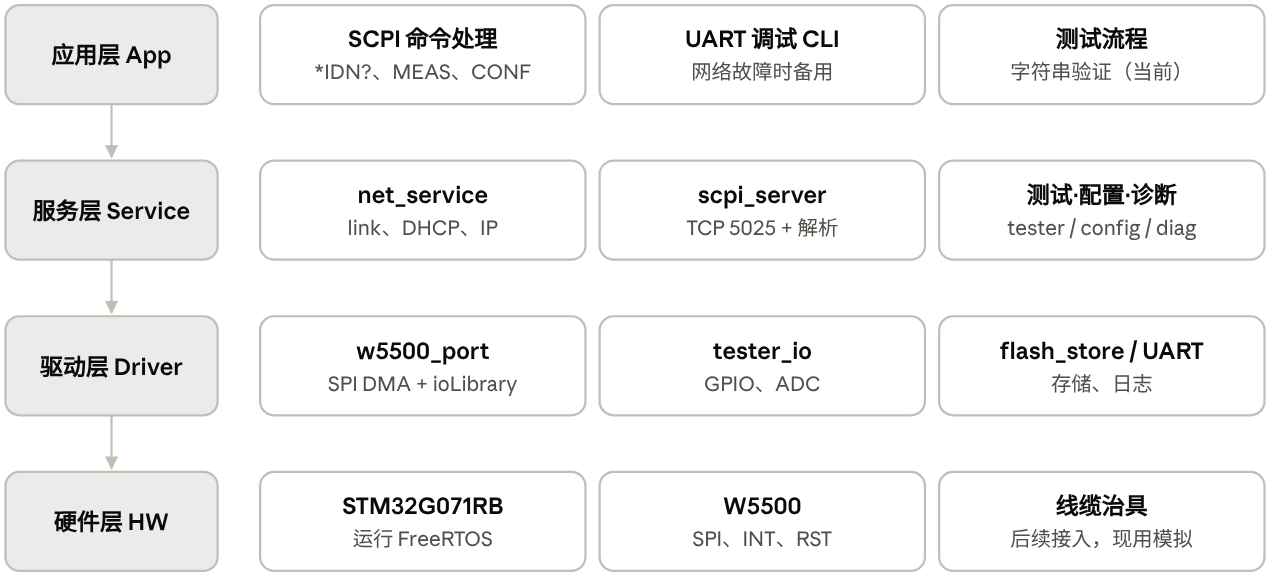
G071RB 注意事项

G071RB 是 Cortex-M0+，比 M4 少了几样东西，设计上做以下调整：

项目	G071RB 的限制	本设计的处理
RAM	36 KB	FreeRTOS heap 12 KB，任务栈按第 6 节缩小，用栈水位实测再调
中断优先级	只有 4 级（0–3），无 BASEPRI	FreeRTOS 临界区会关闭全部中断，临界区要尽量短
DWT 计数器	M0+ 没有 CYCCNT	用 32 位 TIM2 以 1 MHz 计数做延迟测量
Flash	2 KB 一页，按 64 位双字写入	配置存在最后一页 0x0801F800
板载 LED	LD4 在 PA5，与 SPI1 SCK 共用	SPI 通信时 LED 会闪，属正常；状态指示改用其他 GPIO
调试串口	USART2（PA2 / PA3）接 ST-LINK 虚拟串口	log 与备用 CLI 直接走 USB，无需额外 USB 转串口

分层架构

每一层只调用正下方的一层



固件分层架构 · 4 层

规则只有一条：每层只调用正下方一层的头文件接口。App 想知道 SPI 是否正常，必须调用 `diag_service_run_selftest()`，由 service 层再去调 driver，不能直接访问 `w5500_port`。

工程文件夹结构

自己的代码放在 CubeMX 不会覆盖的独立文件夹里，`main.c` 只在 `USER CODE` 区域调用一次 `app_init()`。

```
Project/
├── Core/                  ← CubeMX 生成，不手改
├── Drivers/               ← HAL，不手改
├── ThirdParty/
|   ├── ioLibrary_Driver/  ← WIZnet W5500 库 (socket, DHCP)
|   └── scpi-parser/       ← j123b567/scpi-parser (BSD)
├── L1_Driver/             ← 只 include HAL
|   ├── w5500_port.c/.h
|   ├── tester_io.c/.h
|   └── flash_store.c/.h
├── L2_Service/            ← 只 include L1_Driver
|   ├── net_service.c/.h
|   ├── scpi_server.c/.h
|   ├── tester_service.c/.h
|   ├── config_service.c/.h
|   └── diag_service.c/.h
└── L3_App/                ← 只 include L2_Service
    ├── app_main.c
    └── scpi_commands.c
```

分层由编译器强制：在 CubeIDE 中给每个文件夹单独设置 include path（右键文件夹 → Properties → C/C++ Build → Settings），`L3_App` 看不到 driver 头文件就无法越层调用。CI 中再加一个脚本，检查 `L3_App/` 里是否出现 `#include "w5500` 或 `stm32xx_hal`。

CubeMX 配置

外设	设置（NUCLEO-G071RB）	备注
SPI1	Full-Duplex Master，8-bit，Mode 0（CPOL Low，CPHA 1 Edge），NSS Software；PA5 / PA6 / PA7	起步 8–16 MHz，上限 32 MHz

外设	设置（NUCLEO-G071RB）	备注
SPI1 DMA	DMA1 经 DMAMUX，TX、RX 各一通道，Normal 模式	用于 burst 读写
W5500_CS	GPIO Output，默认高（如 PB0）	软件片选
W5500_RST	GPIO Output，默认高（如 PB1）	复位低电平 >500 µs
W5500_INT	GPIO EXTI，下降沿，上拉（如 PB2）	唤醒 NetTask
FreeRTOS	CMSIS_V2，`configTICK_RATE_HZ = 1000`，`configCHECK_FOR_STACK_OVERFLOW = 2`，heap 12 KB	36 KB RAM 内要留给 ioLibrary 与 SCPI 缓冲
SYS Timebase	TIM6	SysTick 留给 FreeRTOS
TIM2	32 位，预分频到 1 MHz，自由运行	替代 DWT 做 µs 计时
USART2	PA2 / PA3，115200 8N1	ST-LINK 虚拟串口，log 与备用 CLI
NVIC	4 级（0–3）；EXTI、DMA 设 1–2，SysTick / PendSV 保持 3	M0+ 无 BASEPRI，FreeRTOS 临界区全局关中断

CS / RST / INT 引脚为建议值，按你实际接线修改即可。

Driver 层设计

Driver 层只做硬件动作，不含业务判断；每个驱动都提供一个自检函数供 `diag_service` 调用。

W5500 不自己写寄存器协议，直接用 ioLibrary_Driver，只写注册给它的回调：

```
// w5500_port.c
static void cs_sel(void)   { HAL_GPIO_WritePin(W5500_CS_GPIO_Port,
W5500_CS_Pin, GPIO_PIN_RESET); }
static void cs_desel(void) { HAL_GPIO_WritePin(W5500_CS_GPIO_Port,
W5500_CS_Pin, GPIO_PIN_SET); }
static uint8_t spi_rb(void){ uint8_t rx, tx = 0xFF;
HAL_SPI_TransmitReceive(&hspi1, &tx, &rx, 1, 10); return rx; }
static void spi_wb(uint8_t b){ HAL_SPI_Transmit(&hspi1, &b, 1, 10); }
// burst 读写走 DMA，等待期间用 task notification 挂起，不死等
static void spi_rburst(uint8_t *buf, uint16_t len);
```

```
static void spi_wburst(uint8_t *buf, uint16_t len);

int w5500_port_init(void) {
    w5500_hw_reset();                       // RST 拉低 >500us，再等约 2ms
    reg_wizchip_cs_cbfunc(cs_sel, cs_desel);
    reg_wizchip_spi_cbfunc(spi_rb, spi_wb);
    reg_wizchip_spiburst_cbfunc(spi_rburst, spi_wburst);
    uint8_t memsize[2][8] = {{4,4,2,2,1,1,1,1},{4,4,2,2,1,1,1,1}};
    return wizchip_init(memsize[0], memsize[1]);
}

/* 自检接口，给 diag_service 用 */
drv_status_t w5500_selftest(w5500_selftest_t *r) {
    r->version = getVERSIONR();             // 正常必须是 0x04
    uint8_t mac_w[6] = {0x02,0x08,0xDC,0x11,0x22,0x33}, mac_r[6];
    setSHAR(mac_w); getSHAR(mac_r);         // 写读回，验证 SPI 双向
    r->rw_ok   = (memcmp(mac_w, mac_r, 6) == 0);
    r->link_up = (getPHYCFGR() & PHYCFGR_LNK_ON) != 0;
    return (r->version == 0x04 && r->rw_ok) ? DRV_OK : DRV_ERR_SPI;
}
```

读到 `VERSIONR = 0x04` 就说明 SPI 接线、模式和片选都正确；读到 0x00 或 0xFF，先查接线、CS 和 SPI mode。

驱动	对外接口	自检
`w5500_port`	`w5500_port_init()`，ioLibrary 回调	VERSIONR、SHAR 写读回、PHY link
`tester_io`	`tester_io_drive(pin, level)`、`tester_io_read_all()`、`tester_io_adc(pin)`	当前用 tester_io_sim 模拟；有硬件后用短接回环或已知好线缆
`flash_store`	`flash_store_read()`、`flash_store_write()`	写入测试块后校验 CRC

Service 层设计

Service 层把硬件能力变成可用的服务，是 App 层获取任何底层状态的唯一入口。

模块	职责	关键细节
`net_service`	等 PHY link → DHCP → 拿到 IP → 打开 SCPI socket；拔线后回到等 link	DHCP 失败回退到 flash 中的静态 IP；`DHCP_time_handler()` 由 1 s 软件定时器调用
`scpi_server`	TCP 5025 监听，按行收命令交给 scpi-parser	W5500 共 8 个 socket，分 2–3 个给 SCPI 支持多客户端
`tester_service`	逐针驱动、全针读回，判定 open / short / miswire / 导通电阻	结果存入结构体供 `FETCh:RESult?` 读取
`config_service`	网络与测试参数的读写和持久化	flash 最后一页（0x0801F800，2 KB，64 位双字写入），magic + CRC32，损坏则载入默认值
`diag_service`	汇总各层自检、延迟统计	实现 `*TST?` 与 `DIAGnostic` 子系统

`net_service` 状态机：`WAIT_LINK` → `DHCP_RUNNING` → `BOUND`（或 `STATIC_FALLBACK`）→ `SERVING`；任意状态检测到 link down 都回到 `WAIT_LINK`，并关闭所有 socket。

FreeRTOS 任务划分与低延迟

四个任务，由中断和通知驱动而非轮询；W5500 只归 `NetTask` 一个任务所有。

任务	优先级	触发方式	职责	初始栈
`MeasureTask`	Realtime	队列收到测试请求	执行测量，时序要求最严	512 B
`NetTask`	High	W5500 INT → EXTI → task notify	独占 SPI/W5500：link、DHCP、socket 收发	1 KB
`ScpiTask`	AboveNormal	队列收到完整命令行	解析 SCPI，调用 service，回复入发送队列	1.5 KB
`HousekeepTask`	Low	周期 1 s	看门狗、flash 保存、栈水位、异步 log	512 B

降低延迟的措施：

1. W5500 由 `NetTask` 独占。ioLibrary 非线程安全，多任务抢 SPI 加 mutex 容易引起优先级反转。
2. 用 INT 引脚中断 + `vTaskNotifyGiveFromISR` 唤醒，不轮询。
3. SPI burst 走 DMA，传输期间 `ulTaskNotifyTake` 挂起让出 CPU。
4. 任务间通信优先 task notification，其次 queue。
5. M0+ 的 FreeRTOS 临界区会关闭所有中断，临界区和 ISR 都要尽量短，重活交给任务做。
6. 热路径不调用 `printf`，log 经低优先级队列异步输出。
7. 用 `uxTaskGetStackHighWaterMark()` 监控栈余量；RAM 只有 36 KB，按实测调整上表栈大小。

延迟可量化：在 `ScpiTask` 收到命令和发出回复时各读一次 TIM2 计数（1 MHz，G071 没有 DWT），由 `DIAGnostic:LATency?` 返回最近一次处理时间（µs）。

分层测试策略

每层都有板上自检和离线测试两条路径，`*TST?` 一条命令串起全部板上自检。

层	板上测试	离线 / PC 测试	通过标准
Driver	上电跑 `w5500_selftest()`，tester IO 回环	—	VERSIONR = 0x04，写读回一致，回环全通
Service	`DIAGnostic:NET:*` 查看 DHCP、IP、socket 状态	driver 接口做成函数指针结构体，PC 上用 mock driver + Unity 单元测试（如模拟 link down→up 后重新 DHCP）	状态机转移正确，配置 CRC 校验通过
App	UART CLI 执行同一套命令	`pytest` + `pyvisa` 逐条发送 SCPI 并检查回复	所有命令返回预期值，`SYSTem:ERRor?` 为空

UART CLI 与 SCPI 共用同一套命令，网络故障时仍可调试。

SCPI 命令集

命令沿用 IEEE 488.2 公共命令和主流仪器的 `SYSTem:COMMunicate:LAN` 写法，PC 端可直接用 `pyvisa` 以 `TCPIP::<ip>::5025::SOCKET` 连接。

命令	类别	作用 / 返回
`*IDN?`	公共	`Company,CableTester,SN001,FW1.0`
`*RST`	公共	恢复默认状态
`*TST?`	公共	全系统自检，0 = 通过
`SYSTem:ERRor?`	系统	读取错误队列
`DIAGnostic:DRIVer:SPI?`	诊断	W5500 版本与写读回结果
`DIAGnostic:NET:LINK?`	诊断	1 / 0
`DIAGnostic:NET:DHCP?`	诊断	`BOUND` / `FAILED` / `STATIC`
`DIAGnostic:LATency?`	诊断	最近命令处理时间（µs）
`SYSTem:COMMunicate:LAN:DHCP ON|OFF`	网络配置	开关 DHCP
`SYSTem:COMMunicate:LAN:ADDRess "192.168.1.50"`	网络配置	静态 IP
`SYSTem:COMMunicate:LAN:SMASk / :GATeway / :HOSTname`	网络配置	子网、网关、主机名
`SYSTem:COMMunicate:LAN:APPLy`	网络配置	应用网络设置
`SYSTem:CONFig:SAVE`	配置	写入 flash
`CONFigure:PINS 8`	测试配置	芯数
`CONFigure:THReshold:RESistance 2.0`	测试配置	导通电阻阈值（Ω）
`MEASure:CONTinuity?`	测量	运行测试并返回结果
`FETCh:RESult?`	测量	读取上次详细结果

第一阶段：SCPI 字符串验证（暂无测试仪硬件）

在没有测试仪硬件前，用一组纯软件的字符串命令验证整条链路：TCP 连接、按行收发、SCPI 解析、参数处理、错误队列和延迟。字符串逻辑放在 service 层的 `string_service.c`（纯 C，不依赖 HAL），App 层只负责注册 SCPI 回调，分层规则不变。

命令与预期结果

字符串参数上限 128 字节，返回值用双引号包裹（SCPI 字符串格式）。

命令	预期返回	验证点
`TEST:ECHO? "hello"`	`"hello"`	收发与引号解析
`TEST:STRing:LENgth? "abc"`	`3`	整数输出
`TEST:STRing:UPPer? "Ab1"`	`"AB1"`	字符处理
`TEST:STRing:REVerse? "abc"`	`"cba"`	缓冲区处理
`TEST:STRing:CRC? "123456789"`	`"CBF43926"`	CRC32 标准校验值，同一函数之后用于 flash 配置校验
`TEST:STRing:STORe "abc"` 后接 `TEST:STRing:STORe?`	`"abc"`	跨命令保存状态
`TEST:COUNt?`	开机以来处理的命令数	计数与整数输出
`TEST:STRing:LENgth?`（缺参数）	无回复；`SYSTem:ERRor?` 返回 `-109,"Missing parameter"`	错误队列
`TEST:ECHO?` 超过 128 字节	无回复；`SYSTem:ERRor?` 返回 `-223,"Too much data"`	越界保护

实现

```
// L2_Service/string_service.h — 纯 C，可在 PC 上单元测试
size_t   str_svc_upper(const char *in, char *out, size_t cap);
size_t   str_svc_reverse(const char *in, char *out, size_t cap);
uint32_t str_svc_crc32(const char *in, size_t len);
void     str_svc_store(const char *in, size_t len);
size_t   str_svc_fetch(char *out, size_t cap);
```

```
// L3_App/scpi_commands.c — 只调用 service 接口
#define STR_MAX 128

static scpi_result_t TEST_StrUpperQ(scpi_t *ctx) {
    char in[STR_MAX * 2], out[STR_MAX + 1];
    size_t len;
    if (!SCPI_ParamCopyText(ctx, in, sizeof(in), &len, TRUE)) return
SCPI_RES_ERR;
    if (len > STR_MAX) { SCPI_ErrorPush(ctx, SCPI_ERROR_TOO_MUCH_DATA);
return SCPI_RES_ERR; }
    in[len] = '\0';
    str_svc_upper(in, out, sizeof(out));
    SCPI_ResultText(ctx, out);
    return SCPI_RES_OK;
}

const scpi_command_t scpi_commands[] = {
    { .pattern = "*IDN?",               .callback = SCPI_CoreIdnQ },
    { .pattern = "SYSTem:ERRor?",       .callback = SCPI_SystemErrorNextQ },
    { .pattern = "TEST:ECHO?",          .callback = TEST_EchoQ },
    { .pattern = "TEST:STRing:LENgth?", .callback = TEST_StrLenQ },
    { .pattern = "TEST:STRing:UPPer?",  .callback = TEST_StrUpperQ },
    { .pattern = "TEST:STRing:REVerse?",.callback = TEST_StrReverseQ },
    { .pattern = "TEST:STRing:CRC?",    .callback = TEST_StrCrcQ },
    { .pattern = "TEST:STRing:STORe",   .callback = TEST_StrStore },
    { .pattern = "TEST:STRing:STORe?",  .callback = TEST_StrStoreQ },
    { .pattern = "TEST:COUNt?",         .callback = TEST_CountQ },
    SCPI_CMD_LIST_END
};
```

PC 端自动化测试

```
# test_string.py — 运行：pip install pyvisa pyvisa-py pytest，然后 pytest -v
import time, pyvisa, pytest

IP = "192.168.1.200"   # 改成 DHCP 拿到的地址

@pytest.fixture(scope="module")
def dev():
    rm = pyvisa.ResourceManager("@py")
    d = rm.open_resource(f"TCPIP::{IP}::5025::SOCKET")
    d.read_termination = d.write_termination = "\n"
    d.timeout = 2000
    d.write("*CLS")
    yield d
    d.close()

CASES = [
    ('TEST:ECHO? "hello"',           '"hello"'),
    ('TEST:STRing:LENgth? "abc"',    '3'),
    ('TEST:STRing:UPPer? "Ab1"',     '"AB1"'),
    ('TEST:STRing:REVerse? "abc"',   '"cba"'),
    ('TEST:STRing:CRC? "123456789"', '"CBF43926"'),
]

@pytest.mark.parametrize("cmd,expected", CASES)
def test_string(dev, cmd, expected):
    assert dev.query(cmd).strip() == expected
    assert dev.query("SYST:ERR?").startswith("0,")

def test_store_roundtrip(dev):
    dev.write('TEST:STRing:STORe "abc"')
    assert dev.query("TEST:STRing:STORe?").strip() == '"abc"'

def test_missing_param(dev):
    dev.write("TEST:STRing:LENgth?")
    assert dev.query("SYST:ERR?").startswith("-109")

def test_too_long(dev):
    dev.write(f'TEST:ECHO? "{"x" * 200}"')
    assert dev.query("SYST:ERR?").startswith("-223")

def test_latency(dev):
```

```
    n, t0 = 1000, time.perf_counter()
    for _ in range(n):
        assert dev.query('TEST:ECHO? "ping"').strip() == '"ping"'
    rtt_ms = (time.perf_counter() - t0) / n * 1000
    print(f"平均往返 {rtt_ms:.2f} ms，设备内处理
{dev.query('DIAG:LAT?').strip()} us")
```

`test_latency` 同时给出 PC 端往返时间和设备内处理时间，两者之差就是网络与 W5500 收发的开销，可以直接作为给 manager 的延迟数据。

模拟测试仪

硬件到之前，`L1_Driver/tester_io_sim.c` 提供与 `tester_io.h` 相同的接口，模拟一根 8 芯直通线（每根针只与对端同号针导通），由编译开关 `TESTER_IO_SIM=1` 选用。这样 `MEASure:CONTinuity?` 和 `FETCh:RESult?` 现在就能跑通并返回可预期的结果；硬件到位后只换掉这一个文件，service 和 app 层代码不改，这也正好验证了分层设计。

AI 远程调试与安全

设备只提供标准 SCPI over TCP，AI 接入完全在 PC 端完成。在 PC 上写一个小型 Python MCP server，用 `pyvisa` 连接设备，把 `send_scpi(cmd)`、`run_self_test()`、`measure()` 暴露为工具，任何支持 MCP 的 AI 客户端即可操控设备、读取诊断并分析结果。

安全上，LAN 内任何人都能控制设备：建议至少提供可配置的 IP 白名单，并加一个 `SYSTem:LOCK` 机制，避免多人同时操作冲突。

Bring-up 里程碑

按顺序推进，每一步都能单独验证：

- [ ] SPI 读到 `VERSIONR = 0x04`
- [ ] 插网线后 PHYCFGR link 位为 1
- [ ] 设置静态 IP，PC 能 `ping` 通
- [ ] 接入 DHCP，UART 打印获得的 IP
- [ ] TCP 5025 echo，用 `nc <ip> 5025` 验证
- [ ] 接入 scpi-parser，`*IDN?` 正确回复，TEST:STRing 字符串验证全部通过 pytest
- [ ] 迁移到 FreeRTOS 任务，加入 INT 中断和 SPI DMA
- [ ] 先用 tester_io_sim 跑通，有测试仪硬件后实现 tester IO 与 `MEASure` 命令

- [ ] 配置存 flash，`*TST?` 自检，pytest 自动化测试通过

待确认事项

- [ ] STM32 型号：已确认 NUCLEO-G071RB，全文已按其调整
- [ ] 被测线缆芯数与连接器类型，是否需要测电阻（需要 ADC 通道）
- [ ] 是否需要 IP 白名单或密码保护
- [ ] 是否需要设备发现（如 mDNS 或 UDP 广播），方便在 LAN 上找到设备